Dawa Saathi — Complete Deep-Dive Documentation

This document explains the entire bot from scratch to end: how it's set up, how it talks to Telegram, how an image becomes an answer, every service, the database, Redis, and the full flow. Read it top to bottom and you'll be able to explain any part of the system.

1. What this bot does (in one paragraph)

A user sends a photo of a medicine strip to a Telegram bot. The bot reads the label using an AI vision model, and if it is confident it read the medicine correctly, it asks the user *why* they are taking it. Based on that purpose, it generates a simple, elder-friendly awareness summary — common uses, side effects, warnings — and always ends with a disclaimer that this is educational, not medical advice. It never diagnoses, never prescribes, never gives a dosage, and refuses to guess when the photo is unclear.

The single most important design principle: **a confident wrong answer is more dangerous than no answer.** Everything in the architecture serves that principle.

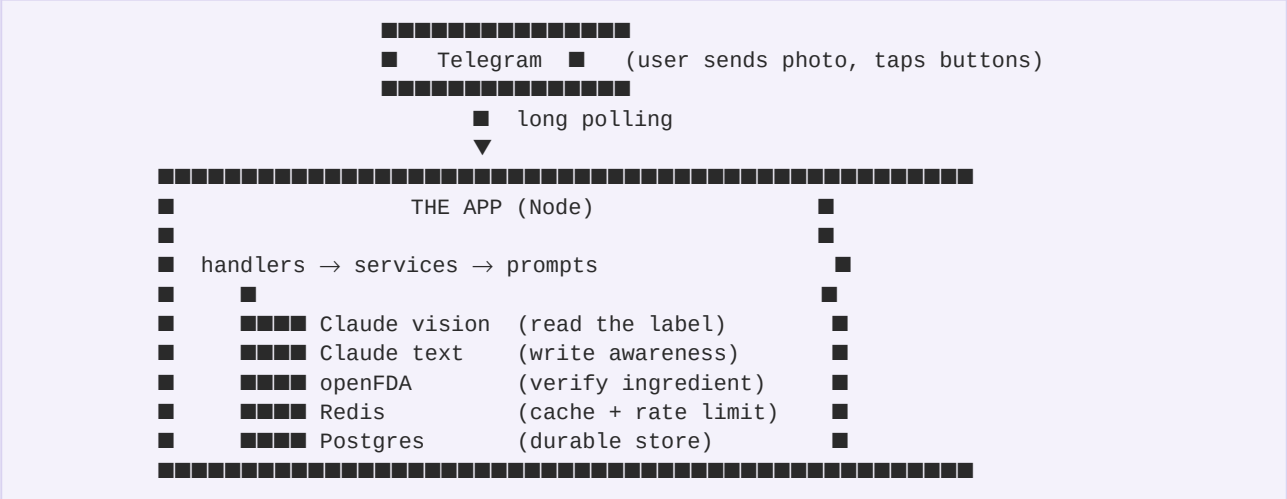
2. The big picture — how the whole thing fits together

There are three running pieces, all started by Docker Compose:

1. **The app** — a Node.js + TypeScript program. This is your code.
2. **PostgreSQL** — a database that permanently stores analyzed medicines and usage history.
3. **Redis** — a fast in-memory store used for caching and rate-limiting.

The app talks to four outside things:

- **Telegram** — to receive photos and send replies
- **Claude (Anthropic API)** — twice: once to read the image, once to write the awareness summary
- **openFDA** — an optional public drug database to verify ingredients
- **Postgres + Redis** — its own data stores



3. Setting up from scratch

3.1 What you need

- **Docker + Docker Compose** — runs everything. The only hard requirement.
- **A Telegram bot token** — free, from @BotFather.
- **An Anthropic API key** — from console.anthropic.com (a few dollars of credit).

3.2 Step by step

Get a Telegram bot token:

1. Open Telegram, search for `@BotFather`
2. Send `/newbot`, pick a name (`Dawa Saathi`) and username (`dawasaathi_bot`)
3. BotFather gives you a token like `8788893205:AAH...` — copy it

Get an Anthropic key:

1. Sign up at console.anthropic.com
2. Add credit, create an API key (starts with `sk-ant-`)

Configure the project:

```
cp .env.example .env
nano .env
```

Fill in:

- `TELEGRAM_BOT_TOKEN` — from BotFather
- `ANTHROPIC_API_KEY` — from Anthropic
- `EXTRACTION_MODEL=claude-sonnet-4-6` (accurate model for reading labels)
- change `POSTGRES_PASSWORD` to anything

Start everything:

```
docker compose up -d --build
docker compose logs -f app
```

When you see `■ Medicine Awareness Bot is live`, open Telegram, find your bot, send `/start`.

4. How the app connects to Telegram

This is the part most people find confusing, so let's go slow. There are **two ways** a Telegram bot can receive messages: **webhooks** and **long polling**. This bot uses **long polling**. Here's the difference.

4.1 Webhooks (what we are NOT using)

With a webhook, *Telegram calls you*. You give Telegram a public URL (like `https://yoursite.com/webhook`). Every time someone messages your bot, Telegram sends an HTTP POST request to that URL with the message inside.

- **Pro:** instant delivery, efficient at high scale
- **Con:** you need a public URL with HTTPS. On your laptop that means ngrok or deploying to a server. Extra setup.

4.2 Long polling (what we ARE using)

With long polling, *you call Telegram*. Your app repeatedly asks Telegram "any new messages for me?" Telegram holds the connection open for up to 25 seconds; if a message arrives in that window it responds

immediately, otherwise it responds empty and your app asks again.

- **Pro:** no public URL needed. Works on localhost, works in Docker, no ngrok.
- **Con:** very slightly less instant, and slightly more chatter. Irrelevant at our scale.

This is configured in `src/services/telegram.service.ts`:

```
export const bot = new TelegramBot(env.TELEGRAM_BOT_TOKEN, {
  polling: {
    interval: 1000,        // ask Telegram every 1 second
    autoStart: false,      // we start it manually after handlers are ready
    params: {
      timeout: 25,         // Telegram holds the connection up to 25s
      allowed_updates: ['message', 'callback_query'],
    },
  },
  request: { family: 4 },  // force IPv4 (Docker IPv6 to Telegram often fails)
});
```

`allowed_updates: ['message', 'callback_query']` tells Telegram we only care about two event types: messages (photos, text) and button taps (callback queries). Without `callback_query` in this list, the purpose buttons would never reach our app.

4.3 The IPv4 detail (why `family: 4`)

Inside Docker, the default network often can't route IPv6 traffic to the internet. Node.js 20 tries IPv6 first by default. When the bot tried to send a message over IPv6, it failed with `EFATAL: AggregateError`. Setting `request: { family: 4 }` forces the Telegram library to use IPv4, which works reliably. This was a real bug we hit and fixed.

4.4 There is no timeout/expiry on the Telegram side for us

Because we use long polling, there's no webhook URL that expires. The bot stays connected as long as the app runs. The only "expiry" in our system is our own: the **pending extraction** in Redis expires after 10 minutes (more on that later).

5. The complete flow — what happens when a user uploads an image

This is the heart of the system. Follow it carefully — every step has a reason.

Step 0 — User sends a photo

The user takes a photo of a medicine strip and sends it in the Telegram chat.

Step 1 — Telegram delivers it to our app

Our long-polling loop receives an "update" containing the photo. Telegram doesn't send the actual image bytes — it sends a `file_id` (a reference) and several size variants. The `photo` event fires in `src/handlers/telegram.handler.ts`.

Step 2 — We pick the largest image and download it

Telegram sends multiple resolutions of every photo. We pick the largest (best for reading small text). Then we call `downloadFile()` which:

1. Asks Telegram for a temporary download URL for that `file_id`
2. Fetches the actual image bytes from that URL

3. Returns them as a Node `Buffer` (raw binary in memory)

This download is wrapped in retry logic — if a network blip happens, it tries 3 times before giving up.

Step 3 — Rate limit check (before any expensive work)

Before doing anything costly, we check: has this user used their 3 free scans today? This uses Redis (explained in section 8). If they're over the limit, we stop here and tell them to come back tomorrow. **We check this first so we never waste an AI call on a user who's over quota.**

Step 4 — Convert image to base64 and send to Claude vision

This answers your question "is it converted to base64?" — **yes.**

The image is raw binary (a Buffer). The Anthropic API accepts images as base64-encoded text. So we convert:

```
const base64 = imageBuffer.toString('base64');
```

Then we send it to Claude with the **extraction prompt** (section 9). Claude looks at the image and returns structured JSON describing what it sees — the medicine name, ingredients, and crucially a `readConfidence` score from 0 to 1.

This is **Stage 1: transcription only.** Claude is told to read *only what is printed*, never to guess or autocorrect a blurry name into a similar-looking medicine.

Step 5 — The confidence gate (the core safety check)

Claude returns its reading plus a confidence score. Now we decide:

- **Not a medicine?** (user sent a selfie, a receipt) → tell them to send a medicine photo. Refund their scan.
- **Confidence below 0.75?** → the photo was too blurry or the name unclear. Ask for a clearer photo. **Refund their scan** so they're not punished for our uncertainty.
- **Confidence 0.75 or above AND a real medicine?** → proceed.

This gate is why the bot will never again confidently say "Mefenamic Acid" when the strip says "Tranexamic Acid." If it's not sure, it stops.

Step 6 — We have a confident reading. Now ASK THE PURPOSE.

This is the key product insight. The same medicine can be prescribed for completely different reasons. Tranexamic Acid is primarily a bleeding-control drug, but dermatologists prescribe it for skin pigmentation. If we just dumped "used to stop bleeding" on a skin patient, they'd panic.

So instead of generating the answer immediately, we:

1. Save the confident reading into Redis (the "pending extraction"), keyed by the user's chat ID, with a 10-minute expiry
2. Send the user a message with tappable buttons: "What are you using this for? ■ Skin / ■ Bleeding / ■ Pain / ..."

The scan quota is consumed at this point (the reading succeeded). The purpose step and the answer generation are free.

Step 7 — User taps a button (or types their reason)

When the user taps a button, Telegram sends a `callback_query` event. Our handler:

1. Immediately removes the buttons (so a second tap can't double-fire)
2. Retrieves and deletes the pending extraction from Redis
3. Maps the button to a purpose string ("skin pigmentation or melasma")
4. Proceeds to generate the answer

If they tap "Type my reason," we wait for their next text message and treat that as the purpose.

Step 8 — Cache lookup (purpose-aware)

Before calling Claude again, we check: have we already generated an answer for *this medicine + this purpose*? The cache key combines the active ingredients and the purpose, so "Tranexamic Acid + skin" and "Tranexamic Acid + bleeding" are cached separately. If we have it cached (Redis, then Postgres), we return it instantly — no AI call, near-zero cost.

Step 9 — openFDA verification (optional enrichment)

If not cached, we optionally query openFDA — a free public US drug database — by the active ingredient. This confirms the drug is real and pulls authoritative purpose/warning text. This is resilient: if openFDA is slow or down, we just skip it and continue. (Note: openFDA is US data, so it matches by *ingredient*, not Indian brand names.)

Step 10 — Claude writes the awareness summary

Now **Stage 2: awareness generation.** We send Claude the confirmed medicine, its ingredients, the user's stated purpose, and any FDA data, using the **awareness prompt** (section 9). Claude returns structured JSON: drug class, how it works, common uses (led by the user's purpose), side effects, serious side effects, warnings per category, interactions, overdose awareness.

Step 11 — Store in cache

We save the generated answer to both Postgres (permanent) and Redis (fast, with a 30-day expiry). Next time anyone scans the same medicine for the same purpose, it's instant.

Step 12 — Format and send to Telegram

We turn the JSON into a clean, readable Telegram message with sections and emoji, append the mandatory disclaimer, and send it (with retry). Done.

Total time: roughly 12 seconds to read + a few seconds to generate = under 20 seconds end to end.

6. The folder architecture (and why it's split this way)

```
src/
■■■ index.ts          Bootstrap: starts everything in the right order, handles shutdown
■■■ app.ts            Express server (only for health checks)
■
■■■ config/           Settings and cross-cutting concerns
■   ■■■ env.ts        Validates every environment variable at startup (Zod)
■   ■■■ logger.ts     Structured JSON logging (Pino)
■   ■■■ constants.ts  The disclaimer text, cache key prefixes, magic numbers
■
■■■ types/
■   ■■■ index.ts      All shared TypeScript types in one place
■
■■■ prompts/          The instructions we give the AI
■   ■■■ extraction.prompt.ts   Stage 1: read the label, never guess
■   ■■■ awareness.prompt.ts    Stage 2: write the simple explanation
■
■■■ services/         The actual work, each file does ONE thing
■   ■■■ telegram.service.ts    Talk to Telegram (receive, download, send)
■   ■■■ vision.service.ts      Stage 1: Claude reads the image
■   ■■■ awareness.service.ts   Stage 2: Claude writes the summary
■   ■■■ fda.service.ts         Verify ingredient against openFDA
■   ■■■ cache.service.ts       Two-tier cache + pending-extraction storage
■   ■■■ ratelimit.service.ts   Daily scan quota
■   ■■■ medicine.service.ts    The ORCHESTRATOR: ties all the above together
■
■■■ handlers/
■   ■■■ telegram.handler.ts    Decides what to do for each Telegram event
■
■■■ routes/
■   ■■■ health.routes.ts       /health and /health/ready endpoints
■
■■■ db/               Database access
■   ■■■ client.ts              Postgres connection pool
■   ■■■ migrate.ts             Creates the tables on startup
■   ■■■ repositories/          Functions that read/write specific tables
■       ■■■ medicine.repository.ts
■       ■■■ usage.repository.ts
■
■■■ redis/
■   ■■■ client.ts              Redis connection
■
■■■ utils/            Small reusable helpers
    ■■■ async.ts               sleep, retry-with-backoff, random IDs
    ■■■ normalize.ts           text hashing, date helpers
    ■■■ format.ts              turns the JSON answer into a Telegram message
```

Why split into so many files? Each file has one job. To change how the AI reads labels, you open `extraction.prompt.ts` and nothing else. To change rate limits, you open `ratelimit.service.ts`. This is "separation of concerns" — it makes the code easy to maintain, easy to test, and easy to explain (like now).

7. The database (PostgreSQL)

7.1 What is a migration?

A migration is code that creates or changes database tables. When the app starts, `src/db/migrate.ts` runs and creates the tables if they don't already exist. It uses `CREATE TABLE IF NOT EXISTS`, so running it many times is safe — it only creates tables the first time.

Why have migrations instead of creating tables by hand? Because the database structure lives *in the code*, version-controlled. Anyone who clones the project and runs it gets the exact right tables automatically. No manual SQL.

7.2 The schema (the three tables)

A "schema" is the shape of your database — what tables exist and what columns they have.

Table 1: `medicines` — the permanent cache of analyzed medicines

```
id              auto-incrementing number (primary key)
ingredient_key  a unique text fingerprint of ingredients+purpose (used for lookup)
medicine_name   the medicine name
analysis        the full AI answer, stored as JSONB (JSON in the database)
hit_count       how many times this medicine has been looked up
created_at      when first stored
updated_at      when last updated
```

Table 2: `usage_log` — tracks how many scans each user did each day

```
id                auto number
telegram_user_id  who
scan_date         which day (YYYY-MM-DD)
scan_count        how many scans that day
UNIQUE (telegram_user_id, scan_date)   one row per user per day
```

Table 3: `scan_log` — analytics: every scan attempt and its outcome

```
id                auto number
telegram_user_id  who
outcome           what happened (extraction_ok, low_confidence, etc.)
cache_hit         was it served from cache?
read_confidence   how confident the vision read was
duration_ms       how long it took
created_at        when
```

7.3 What is an index?

An index is like the index at the back of a book — it lets the database find rows fast without scanning every row. We create indexes on the columns we search by often:

- `idx_medicines_name` — fast lookup by medicine name
- `idx_usage_user_date` — fast lookup of "how many scans did user X do today"
- `idx_scan_log_created` — fast sorting of analytics by time

Without an index, finding one row in a million means checking all million. With an index, it's near-instant.

7.4 What is JSONB?

`JSONB` is a Postgres column type that stores JSON in an efficient binary form. We store the entire AI answer (a complex nested object) in a single `analysis` column. This is simpler than creating 15 separate columns for uses, side effects, warnings, etc. When we read it back, Postgres hands us the object directly.

8. Redis — what we store and why

Redis is an in-memory key-value store. It's extremely fast (microseconds) but data can expire. We use it for two things.

8.1 Caching medicine answers (speed + cost saving)

When we generate an answer, we store it in Redis with a 30-day expiry, keyed by `med:v1:<ingredient+purpose fingerprint>`. Next time someone scans the same medicine for the same purpose, we find it in Redis instantly and skip the expensive Claude call.

Redis is **tier 1** (fast but expires). Postgres is **tier 2** (permanent). The lookup order is: Redis first, then Postgres, then (if both miss) call Claude. When Postgres has it but Redis doesn't, we copy it back into Redis so it's fast next time. This is a classic two-tier cache.

8.2 Rate limiting (the daily scan quota)

Each user gets 3 free scans per day. We track this with a Redis counter keyed by `rl:daily:v1:<userId>:<date>`. Every scan does an atomic `INCR` (increment) on this key. If the count goes above 3, the user is blocked until tomorrow.

The clever part: the key is set to expire at midnight. So at midnight the counter automatically vanishes and everyone gets a fresh 3 scans. No cleanup job needed.

We also "refund" a scan (decrement the counter) when the scan fails for a reason that isn't the user's fault — like a blurry photo. The user shouldn't lose a scan because our vision wasn't sure.

8.3 Pending extraction (the purpose-question state)

Between "we read the medicine" and "user tells us why they take it," we need to remember the reading. We store it in Redis keyed by `pending:v1:<chatId>` with a 10-minute expiry. When the user taps a purpose button, we retrieve and delete it. If they wait more than 10 minutes, it expires and they're asked to re-send the photo. This is why there's a time limit — it's our own, and it's 10 minutes.

9. The two prompts (the brain of the system)

A "prompt" is the instruction we give the AI. We have two, one for each stage. Splitting them is itself a safety decision: the AI never reads-and-interprets in a single muddy step.

9.1 The extraction prompt (`extraction.prompt.ts`)

Job: read what is literally printed on the label. Nothing else.

Key rules baked into it:

- "You are a careful transcriptionist, not a doctor"
- "Read ONLY what is printed. Never autocorrect or guess."
- "If the name is unclear, score your confidence LOW rather than guessing"
- It explicitly warns about look-alike drug names (the Tranexamic vs Mefenamic trap)
- Returns JSON with `medicineName`, `activeIngredients`, `isMedicine`, and `readConfidence` (0 to 1)

This prompt is why the bot stopped giving wrong answers. By forcing the AI to *only transcribe* and *score its confidence honestly*, we catch uncertain reads before they become dangerous answers.

9.2 The awareness prompt (`awareness.prompt.ts`)

Job: turn a confirmed medicine into a simple, safe explanation.

Key rules baked into it:

- Absolute prohibitions: never diagnose, never prescribe, never give a dosage, never say "safe to take"

- Write in simple words an elderly person understands ("stomach" not "gastric")
- The purpose rule: if the user said why they take it, LEAD with that use so they aren't frightened by an unrelated primary use
- Base the answer ONLY on the confirmed ingredients — don't invent information
- Returns structured JSON (drug class, uses, side effects, serious side effects, warnings, interactions, overdose awareness)

The purpose rule is what produced your correct experience: scanning Tranexamic Acid and tapping "skin" led the answer with the pigmentation use, calmly, instead of alarming you with "stops bleeding."

10. Every service explained

10.1 `telegram.service.ts`

Owns the connection to Telegram. Creates the bot (long polling, IPv4), and provides:

- `startBot()` / `stopBot()` — lifecycle
- `downloadFile()` — turns a Telegram `file_id` into image bytes (with retry)
- `sendMarkdownMessage()` — sends a formatted reply, splitting it if too long
- `sendMessageWithRetry()` — sends any message with 3 retries (fixes network blips)
- `sendTypingAction()` — shows "typing..." so the user knows we're working

10.2 `vision.service.ts` (Stage 1)

Takes the image buffer, converts it to base64, sends it to Claude with the extraction prompt, parses the JSON reply, and returns the reading with its confidence. Uses the accurate Sonnet model because reading correctly is safety-critical. Has retry for transient API errors.

10.3 `awareness.service.ts` (Stage 2)

Takes the confirmed medicine + the user's purpose + any FDA data, sends it to Claude with the awareness prompt, parses and validates the JSON, and returns the structured analysis. Uses the cheaper Haiku model because by now the medicine is confirmed; we just need clear writing.

10.4 `fda.service.ts`

Queries the free openFDA API by active ingredient to verify the drug exists and pull authoritative purpose/warning text. Fully resilient — any failure is caught and we continue without it. It's a *secondary* accuracy aid, not the primary safety (that's vision + the confidence gate).

10.5 `cache.service.ts`

The two-tier cache (Redis → Postgres) plus the pending-extraction storage. Provides `lookupCachedMedicine()`, `storeMedicine()`, `savePendingExtraction()`, `takePendingExtraction()`.

10.6 `ratelimit.service.ts`

The daily quota. Provides `checkAndReserveScanSlot()` (atomic Redis increment with midnight expiry), `rollbackScanSlot()` (refund on no-fault failures), `recordScanInDb()` (durable analytics), and `getCurrentUsage()` (for the /usage command).

10.7 `medicine.service.ts` (the orchestrator)

This is the conductor. `runMedicinePipeline()` runs Stage 1: rate limit → size check → vision extraction → confidence gate, and returns the confirmed extraction (stopping there so the handler can ask the purpose). `generateAwarenessForExtraction()` runs Stage 2: cache lookup → FDA enrich → awareness generation → store. Splitting the pipeline at the purpose question is the key structural decision.

11. The handler explained (`telegram.handler.ts`)

The handler decides what to do for each kind of Telegram event. It registers listeners:

- `/start`, `/help`, `/usage` — text commands, each sends a canned reply
- `photo` and `document` — runs Stage 1, then asks the purpose
- `callback_query` — fires when a purpose button is tapped; removes the buttons, retrieves the pending extraction, runs Stage 2
- `message` (text) — if the user has a pending extraction, treats their text as the typed purpose; otherwise gently asks for a photo

The key helpers:

- `processImage()` — download → pipeline → on success, save pending + ask purpose
- `askPurpose()` — builds the tappable button keyboard and sends the question
- `handleCallbackQuery()` — handles button taps (with double-tap protection)
- `runAwarenessAndReply()` — shared: generate the answer and send it

The double-tap protection: when a button is tapped, we immediately edit the message to remove the buttons, so the user physically cannot tap twice. If a tap somehow arrives with no pending data, we stay silent rather than showing a confusing "expired" message.

12. Bootstrap and shutdown (`index.ts`)

Startup order matters and is deliberate:

1. Connect to Postgres, run migrations (tables must exist before anything)
2. Connect to Redis
3. Start the Express HTTP server (health checks go live)
4. Register Telegram handlers, then start polling (handlers must exist before messages arrive)
5. Install signal handlers for graceful shutdown

Graceful shutdown: on Ctrl+C or `docker stop`, the app stops accepting new messages, finishes what it's doing, then closes Redis and Postgres cleanly. Inside Docker, `tini` forwards the stop signal correctly.

13. The safety architecture (the part to emphasize in a speech)

Safety is enforced at FOUR independent layers. Even if one fails, the others hold.

1. **The split pipeline** — reading and interpreting are separate steps. The AI never guesses-and-explains in one muddy pass.
2. **The extraction prompt** — explicitly forbids guessing, warns about look-alike names, demands an honest confidence score.

3. **The confidence gate** — below 0.75, we refuse to answer and refund the scan. No confident wrong answers.

4. **The awareness prompt + disclaimer** — never diagnoses, prescribes, or gives dosage; the disclaimer is appended in code (not trusted to the AI) on every single message.

The guiding principle, again: **a confident wrong answer is worse than no answer.** In a medicine app, that is the only acceptable stance.

14. Cost per scan (for the speech)

- Vision read (Claude Sonnet): a few rupees
- Awareness write (Claude Haiku): under a rupee
- Cache hit: effectively free (Redis/Postgres only)
- Telegram, openFDA: free

A repeat scan of a known medicine costs almost nothing because of the cache. A fresh scan costs a few rupees. At scale, the cache hit rate keeps average cost very low.

15. One-sentence summary for each question you asked

- **How is it set up?** Docker Compose runs the app + Postgres + Redis; you only supply two API keys.
- **How does it integrate with Telegram?** Long polling — the app asks Telegram for messages every second, no public URL needed.
- **How do webhooks work / what's the time limit?** We don't use webhooks; the only expiry is our own 10-minute pending-extraction window in Redis.
- **How is the image extracted / is it base64?** Yes — the image bytes are base64-encoded and sent to Claude vision, which reads the label.
- **How does the database work?** Postgres with three tables, created by migrations on startup, with indexes for fast lookup and JSONB for storing answers.
- **How does Redis work?** Fast tier-1 cache, daily rate-limit counters that auto-expire at midnight, and 10-minute pending-extraction state.
- **What does the handler do?** Decides what to do for each Telegram event and runs the two-stage flow.
- **What's the extraction prompt?** Stage 1: read the label, never guess, score confidence honestly.
- **What's the awareness prompt?** Stage 2: write a simple, safe explanation led by the user's stated purpose.
- **What are the services?** Each does one job: Telegram, vision, awareness, FDA, cache, rate limit, and the orchestrator that ties them together.